



Module 14 — Data Cleaning robuste

Bootcamp Data Analyst — From Zero to Hero | Niveau Intermédiaire · Module 14



Ce que tu seras capable de faire à la fin de ce module

- Nettoyer des données textuelles complexes avec des **expressions régulières**, en Python et en R
- Extraire plusieurs informations d'une même chaîne de texte (groupes de capture)
- Lire une erreur Python (traceback) et un avertissement R, et savoir où chercher le problème
- Connaître le piège du **NA silencieux** en R — très différent du crash immédiat de Python
- Isoler un bug avec une méthode simple et reproductible
- Documenter un script proprement : docstrings Python, roxygen2 en R, README



Durée estimée : 2 heures  **Outils** : Python (module `re`) · R (`stringr`, `dplyr`) 

Prérequis : Module 13 terminé · Modules 10-12 (Python & R) du Niveau Débutant

1. Pourquoi aller au-delà de

`SUPPRESpace` / `.str.strip()` / `str_trim()`

Au Niveau Débutant, tu as déjà nettoyé des données : espaces parasites, valeurs manquantes. Mais les vraies données administratives et commerciales ivoiriennes posent des problèmes plus tordus :

DONNÉE BRUTE	LE PROBLÈME
"07 12 34 56 78", "0712345678", "+225 0712345678", "225-07-12-34-56-78"	Un même numéro de téléphone, 4 formats différents
"1 250 000", "1.250.000", "1250000 FCFA"	Un même montant, séparateurs de milliers différents
"07/12/2023", "07-12-2023", "7 décembre 2023"	Une même date, 3 formats différents
"Kouassi, Jean (07-12-2023)"	Plusieurs informations mélangées dans une seule cellule texte

`.str.strip()` / `str_trim()` ne résout aucun de ces cas. Pour tout le reste, il faut décrire un **motif** (pattern) — c'est exactement ce que fait une expression régulière (regex), disponible aussi bien en Python qu'en R.

2. Les expressions régulières (regex) — Python & R

Une regex est un texte qui décrit un **motif** à rechercher. Le langage des motifs (`\d`, `\s`, `+`, `*`, `[...]` ...) est **identique en Python et en R** — seules les fonctions qui l'utilisent changent de nom.

Les briques essentielles (communes aux deux langages)

MOTIF	SIGNIFIE	EXEMPLE
<code>\d</code>	Un chiffre	<code>\d\d</code> matche <code>"123"</code>
<code>\D</code>	Tout sauf un chiffre	
<code>\s</code>	Un espace (space, tab...)	
<code>+</code>	1 fois ou plus	<code>\d+</code> matche <code>"7"</code> , <code>"12"</code> , <code>"345678"</code>
<code>*</code>	0 fois ou plus	
<code>?</code>	0 ou 1 fois	
<code>[...]</code>	Un caractère parmi une liste	<code>[A-Za-z]</code> = une lettre
<code>^</code> / <code>\$</code>	Début / fin de chaîne	
<code>(...)</code>	Un groupe de capture (pour extraire un morceau précis)	

Cas 1 — Uniformiser des numéros de téléphone ivoiriens

Python :

```
PYTHON
import re

numeros = ["07 12 34 56 78", "0712345678", "+225 0712345678", "225-07-12-34-56-78"]

def uniformiser(numero):
    chiffres = re.sub(r"\D", "", numero)  # ne garder que les chiffres
    return chiffres[-10:]                # garder les 10 derniers (retire le 225 éventuel)

propres = [uniformiser(n) for n in numeros]
print(propres)
# ['0712345678', '0712345678', '0712345678', '0712345678']
```

R (équivalent avec `stringr`) :

R

```
library(stringr)

numeros <- c("07 12 34 56 78", "0712345678", "+225 0712345678", "225-07-12-34-56-78")

uniformiser <- function(numero) {
  chiffres <- str_remove_all(numero, "\\D") # ne garder que les chiffres
  str_sub(chiffres, -10, -1)                # garder les 10 derniers
  caracteres
}

propres <- sapply(numeros, uniformiser)
print(propres)
# [1] "0712345678" "0712345678" "0712345678" "0712345678"
```

💡 En R, une regex à l'intérieur d'une chaîne de caractères s'écrit avec un **double backslash** (`"\\D"`) — le premier backslash échappe le second pour que R transmette bien `\\D` au moteur de regex. Même motif qu'en Python, syntaxe d'échappement différente.

Cas 2 — Extraire un montant

Python :

PYTHON

```
montant = "1 250 000 FCFA"
chiffres = re.sub(r"^[^\\d]", "", montant)
print(int(chiffres))
# 1250000
```

R :

R

```
montant <- "1 250 000 FCFA"
chiffres <- str_remove_all(montant, "[^0-9]")
as.numeric(chiffres)
# [1] 1250000
```

Cas 3 — Extraire plusieurs informations d'une même chaîne (groupes de capture)

Reprenons `"Kouassi, Jean (07-12-2023)"` de la section 1 — comment en extraire le nom, le prénom et la date séparément ?

Python :

PYTHON

```
texte = "Kouassi, Jean (07-12-2023)"
match = re.match(r"(\w+), (\w+) \((\d{2})-(\d{2})-(\d{4})\)", texte)
nom, prenom, jour, mois, annee = match.groups()
print(f"{prenom} {nom}, né(e) le {annee}-{mois}-{jour}")
# Jean Kouassi, né(e) le 2023-12-07
```

R (avec `str_match()`) :

R

```
texte <- "Kouassi, Jean (07-12-2023)"
resultat <- str_match(texte, "(\\w+), (\\w+) \\((\\d{2})-(\\d{2})-(\\d{4})\\)")
nom <- resultat[1, 2]
prenom <- resultat[1, 3]
jour <- resultat[1, 4]
mois <- resultat[1, 5]
annee <- resultat[1, 6]
cat(prenom, nom, ", né(e) le", annee, "-", mois, "-", jour, "\n")
# Jean Kouassi , né(e) le 2023 - 12 - 07
```

 **Pont** : chaque `(...)` dans le motif devient un élément de `match.groups()` en Python, ou une colonne de la matrice retournée par `str_match()` en R (la colonne 1 est toujours la chaîne complète, les groupes commencent à la colonne 2).

👉 À toi de jouer

Écris une fonction (en Python ou en R) qui uniformise ces 3 dates dans un même format `AAAA-MM-JJ` : `"07/12/2023"`, `"07-12-2023"`, `"7 décembre 2023"`. Indice : il te faudra deux motifs différents (un pour les formats numériques avec `/` ou `-`, un pour le format en toutes lettres) et un petit dictionnaire `{"décembre": "12", ...}` pour convertir le nom du mois.

3. Déboguer un script qui plante — Python & R

Python : un crash immédiat et explicite

PYTHON

```
import pandas as pd

df = pd.DataFrame({"salaire": ["450000", "380 000", "non renseigné"]})
df["salaire_num"] = df["salaire"].astype(int)
```

```
Traceback (most recent call last):
  File "script.py", line 3, in <module>
    df["salaire_num"] = df["salaire"].astype(int)
ValueError: invalid literal for int() with base 10: '380 000'
```

Trois informations essentielles, à lire **de bas en haut** :

1. Le type d'erreur (`ValueError`) et son message : `int()` ne sait pas convertir `'380 000'` (l'espace bloque la conversion)
2. La ligne exacte qui a planté (`line 3`)
3. La pile d'appels au-dessus, si l'erreur vient d'une fonction imbriquée

R : le piège du NA silencieux

Le même problème en R se comporte très différemment — **et c'est un piège classique** :

R

```
df <- data.frame(salaire = c("450000", "380 000", "non renseigné"))
df$salaire_num <- as.integer(df$salaire)
```

```
Warning message:
NAs introduced by coercion
```

R **ne plante pas**. Il affiche un simple avertissement et continue, en remplaçant silencieusement `"380 000"` ET `"non renseigné"` par `NA` — sans distinguer "une vraie valeur mal formatée" d'une "valeur légitimement absente". Si tu ne lis pas tes avertissements, ton script continue avec des données fausses sans que rien ne t'alerte.

⚠ **Réflexe R indispensable** : `options(warn = 2)` transforme tous les avertissements en erreurs bloquantes — utile pendant le développement pour ne rater aucun avertissement silencieux comme celui-ci.

Méthode pour isoler un bug (valable dans les deux langages)

1. **Reproduis en petit** : teste la ligne fautive sur une seule valeur (`int("380 000")` ou `as.integer("380 000")`) plutôt que sur tout un jeu de données.
2. **Affiche l'état intermédiaire** : `df["salaire"].unique()` (Python) ou `unique(df$salaire)` (R) révèle souvent le format inattendu.
3. **Corrige à la source** avec une regex, plutôt que de bricoler un `try/except` (Python) ou `tryCatch` (R) qui cache le problème.

Python (correction) :

```
PYTHON
df["salaire_num"] = df["salaire"].apply(
    lambda s: int(re.sub(r"\D", "", s)) if re.search(r"\d", s) else None
)
print(df)
#           salaire  salaire_num
# 0         450000      450000.0
# 1         380 000      380000.0
# 2   non renseigné           NaN
```

R (correction) :

```
R
library(dplyr)
library(stringr)

df <- df %>%
  mutate(salaire_num = if_else(
    str_detect(salaire, "\\d"),
    as.integer(str_remove_all(salaire, "\\D")),
    NA_integer_
  ))
print(df)
#           salaire salaire_num
# 1         450000      450000
# 2         380 000      380000
# 3 non renseigné         NA
```

💡 Un `try/except` / `tryCatch` qui avale l'erreur sans rien afficher est le pire réflexe de debugging, dans les deux langages — tu perds l'information qui t'aurait dit où est le problème.

4. Documenter son travail — Python & R

Un script qui fonctionne aujourd'hui et qu'on ne comprend plus dans 3 mois n'est pas un script fini.

Docstring Python

PYTHON

```
def uniformiser_numero(numero: str) -> str:
    """Uniformise un numéro de téléphone ivoirien en 10 chiffres sans
    préfixe.

    Gère les formats : espaces, tirets, +225, préfixe 225.
    Exemple : "+225 07 12 34 56 78" -> "0712345678"
    """
    chiffres = re.sub(r"\D", "", numero)
    return chiffres[-10:]
```

Documentation roxygen2 en R

R utilise une convention équivalente, avec des commentaires préfixés par `#'` :

R

```
#' Uniformise un numéro de téléphone ivoirien en 10 chiffres sans préfixe
#'
#' Gère les formats : espaces, tirets, +225, préfixe 225.
#'
#' @param numero Un numéro de téléphone sous forme de texte
#' @return Le numéro nettoyé, 10 chiffres, sans préfixe
#' @examples
#' uniformiser_numero("+225 07 12 34 56 78") # "0712345678"
uniformiser_numero <- function(numero) {
  chiffres <- str_remove_all(numero, "\\D")
  str_sub(chiffres, -10, -1)
}
```

Commentaires utiles vs bruit (les deux langages)

PYTHON

```
# ❌ Commentaire inutile – répète ce que le code dit déjà
total = df["montant"].sum() # calcule la somme de montant

# ✅ Commentaire utile – explique un choix non-évident
total = df["montant"].sum() # les valeurs négatives = remboursements, on les
garde volontairement
```

Un bon commentaire répond à "pourquoi ?", jamais à "quoi ?" — le code répond déjà à "quoi", en Python comme en R.

README minimal pour un projet d'analyse

MARKDOWN

```
# Analyse RH – Bamba & Associés

## Contenu
- `nettoyage.py` / `nettoyage.R` – uniformise les numéros de téléphone et montants
- `analyse.py` / `analyse.R` – calculs et graphiques

## Comment l'exécuter
1. `pip install -r requirements.txt` (Python) ou
   `install.packages(c("stringr","dplyr"))` (R)
2. Exécuter le script de nettoyage puis celui d'analyse

## Données
Source : export RH interne (confidentiel, non inclus dans ce dépôt)
```

C'est le même réflexe que le README GitHub vu au Module 13 — c'est souvent la première chose qu'un·e collègue ou un·e recruteur·se lit, quel que soit le langage utilisé derrière.

5. Pont Python ↔ R — data cleaning et debugging

OPÉRATION	PYTHON	R
Chercher un motif (booléen)	<code>re.search(pattern, texte)</code>	<code>str_detect(texte, pattern)</code>
Remplacer tout ce qui matche	<code>re.sub(pattern, repl, texte)</code>	<code>str_remove_all(texte, pattern)</code> / <code>str_replace_all(...)</code>
Extraire des groupes de capture	<code>re.match(pattern, texte).groups()</code>	<code>str_match(texte, pattern)</code> (colonnes 2+)
Garder les N derniers caractères	<code>texte[-10:]</code>	<code>str_sub(texte, -10, -1)</code>
Convertir un texte non-numérique en entier	<code>int("abc")</code> → crash immédiat (<code>ValueError</code>)	<code>as.integer("abc")</code> → <code>NA</code> + avertissement silencieux
Rendre les avertissements bloquants	(déjà bloquant par défaut)	<code>options(warn = 2)</code>
Documenter une fonction	Docstring <code>"""..."""</code>	Commentaires roxygen2 <code>#' ...</code>
Éviter d'avaler une erreur silencieusement	Ne jamais faire un <code>except:</code> <code>pass</code> nu	Ne jamais faire un <code>tryCatch(..., error = function(e) NULL)</code> nu

🔗 Retiens surtout la différence de philosophie : **Python crashe fort et tôt** (tu ne peux pas l'ignorer), **R avertit et continue** (tu peux facilement rater le problème si tu ne lis pas la console). Aucune des deux approches n'est "meilleure" — mais tu dois savoir laquelle tu utilises pour adapter ta vigilance.

✅ Résumé du module

CONCEPT	CE QU'IL FAUT RETENIR
Regex	Décrit un motif à rechercher/remplacer — même langage de motifs en Python et en R
<code>\d</code> , <code>\s</code> , <code>+</code> , <code>*</code> , <code>[...]</code> , <code>(...)</code>	Briques de base, identiques dans les deux langages
<code>re.search</code> / <code>str_detect</code>	Vérifier qu'un motif existe dans un texte
<code>re.sub</code> / <code>str_remove_all</code>	Remplacer ce qui matche un motif
<code>re.match(...).groups()</code> / <code>str_match</code>	Extraire plusieurs informations d'une même chaîne
Traceback Python	Se lit de bas en haut : type d'erreur, message, ligne fautive — crash immédiat
<code>NA</code> silencieux en R	<code>as.integer()</code> sur du texte mal formaté ne plante pas, il avertit et continue — piège classique
<code>options(warn = 2)</code>	Transforme les avertissements R en erreurs bloquantes
Isoler un bug	Reproduire en petit, afficher l'état intermédiaire, corriger à la source
<code>try/except</code> / <code>tryCatch</code> silencieux	Le pire réflexe, dans les deux langages
Docstring / roxygen2	Explique ce que fait une fonction et comment l'utiliser
Bon commentaire	Répond à "pourquoi", jamais à "quoi"
README	Contenu, comment exécuter, source des données

Quiz — Vérifie ta compréhension

Q1. Quelle regex retire tout ce qui n'est pas un chiffre d'une chaîne `numero` en Python ?

- a) `re.sub(r"\d", "", numero)`
- b) `re.sub(r"\D", "", numero)`

- c) `re.search(r"\d+", numero)`

👉 Voir la réponse

✅ b) — `\D` (D majuscule) signifie "tout sauf un chiffre". `re.sub(r"\D", "", numero)` remplace chaque caractère non-numérique par rien, ne laissant que les chiffres. La réponse a) ferait l'inverse (supprimerait les chiffres), et c) ne fait que chercher, pas remplacer.

Q2. En R, `as.integer("380 000")` renvoie...

- a) Une erreur qui arrête le script, comme en Python
- b) `NA`, avec un simple avertissement — le script continue
- c) `380000`, R retire automatiquement les espaces

👉 Voir la réponse

✅ b) — Contrairement à Python qui lève une `ValueError` bloquante, R affiche `"NAs introduced by coercion"` et continue silencieusement avec un `NA`. C'est pour ça qu'il faut toujours lire ses avertissements en R, ou utiliser `options(warn = 2)` pour les transformer en erreurs.

Q3. Ton script plante avec `ValueError: invalid literal for int() with base 10: '380 000'`. Que fais-tu en premier ?

- a) J'entoure la ligne d'un `try/except: pass` pour que le script continue
- b) Je teste `int("380 000")` isolément pour comprendre exactement ce qui bloque
- c) Je relance le script en espérant que ça passe cette fois

👉 Voir la réponse

✅ b) — Reproduire le problème en isolant la valeur fautive te montre immédiatement que c'est l'espace qui bloque `int()`. Le `try/except: pass` (a) cache l'erreur sans la résoudre — tu auras juste une valeur manquante inexpliquée plus loin.

Q4. Tu as `texte <- "Kouassi, Jean (07-12-2023)"` en R et le motif `"(\\w+), (\\w+) \\\n(\\d{2})-(\\d{2})-(\\d{4})\\)"`. Comment récupères-tu le prénom ("Jean") après `resultat <- str_match(texte, motif)` ?

- a) `resultat[1, 1]`
- b) `resultat[1, 2]`
- c) `resultat[1, 3]`

👉 Voir la réponse

✅ c) — La colonne 1 de `str_match()` est toujours la chaîne complète qui matche. La colonne 2 est le premier groupe `(\\w+)` (le nom, "Kouassi"), la colonne 3 est le deuxième groupe (le prénom, "Jean").

Q5. Lequel de ces commentaires est utile ?

- a) `# incrémente x de 1` au-dessus de `x += 1`
- b) `# on exclut les stagiaires : leur contrat n'a pas de date de fin, ça fausserait la moyenne` au-dessus d'un filtre
- c) `# fonction qui calcule quelque chose`

👉 Voir la réponse

✅ b) — Ce commentaire explique une décision non-évidente (*pourquoi* exclure les stagiaires) que le code seul ne révèle pas. Les options a) et c) répètent ou vagalisent ce que le code dit déjà.

➡ Module suivant

Tu sais maintenant nettoyer des données réelles et déboguer proprement, dans les deux langages. Dans le **Module 15**, on approfondit pandas et on découvre DuckDB pour interroger des DataFrames en SQL.

